



Clase 1: Fundamentos de Python para Data Engineering (ETL)

Expositor: Angel Tintaya

Objetivo de la sesión: Aprender el vocabulario base de Python. Antes de crear bots o procesar millones de registros, necesitamos entender cómo la computadora guarda, lee y transforma la información.

1. Variables: Las Cajas de Almacenamiento

¿Qué es una **variable**? Imagina que tienes una caja de cartón, le pegas una etiqueta con un nombre y guardas algo dentro. En Python, la "etiqueta" es el nombre de la variable y "lo que guardas" es el dato.

Python

```
# Si intentamos llamar a una "caja" que no hemos creado, Python lanza un error.  
resultado # Variable no definida (Sale Error porque no se ha definido)
```

```
# Declarando variables: Guardamos el número 2500 en la caja 'monto_deuda'  
monto_deuda = 2500  
# Guardamos texto (string) en la caja 'moneda'  
moneda = 'Soles'
```

Interactuando con el usuario:

La función `input()` hace que el programa se pause y le pida al usuario que escriba algo. Todo lo que el usuario escribe, Python lo lee como texto. Si necesitamos que sea un número para hacer matemáticas, usamos `int()` (que significa *integer* o entero) para transformarlo.

Python

```
monto_interes = int(input('Ingrese el interes: '))
```

Operaciones y Tipos de Datos:

Podemos hacer matemáticas básicas (como sumar con `+` o multiplicar con `*`). Para saber qué tipo de dato hay dentro de nuestra "caja", usamos la función `type()`.

Python

```
resultado = monto_deuda + monto_interes  
resultado # Mostramos el resultado
```

```
# type() nos dirá si es un 'int' (número entero) o un 'str' (texto)  
type(monto_deuda)  
type(moneda)
```

```
# Operando con variables
```

```
4 * monto_deuda
```

```
# Transformando tipos de datos (Casteo)
```

```
# A veces necesitamos que un número se comporte como texto para unirlo a una frase
```

```
str_monto_deuda = str(monto_deuda)
```

```
str_monto_deuda
```

```
# Y viceversa, convertir texto a número para sumar
```

```
int_monto_deuda = int(str_monto_deuda)
```

```
int_monto_deuda
```

2. Strings: Las Cadenas de Texto

Un *string* es cualquier cosa que esté entre comillas. En Datos, pasaremos mucho tiempo limpiando textos (nombres mal escritos, direcciones desordenadas).

Creación y Concatenación:

```
Python
```

```
# El signo '+' en textos no suma matemáticamente, los "pega" (concatena)
```

```
"a" + "b"
```

```
# Podemos usar comillas simples o dobles.
```

```
# Si el texto ya tiene una comilla simple (como en el inglés It's), usamos dobles por fuera para no confundir a Python.
```

```
text_1 = "It's a string with a single quote"
```

```
# El símbolo '\' (backslash) funciona como un "escudo". Le dice a Python: "La siguiente comilla es parte del texto, no es el final del código".
```

```
text_2 = 'It\'s a string with a single quote'
```

```
# Las comillas triples sirven para escribir textos de varias líneas o textos que ya contienen muchas comillas por dentro.
```

```
text_3 = """He said: "Hello, I've got a question" from the audience"""
```



Métodos de String (Limpieza de Datos)

¿Qué es un método (el punto .)? Imagina que el texto es un control remoto y el punto . es presionar un botón para que haga algo específico.

```
Python
```

```
string_1 = 'hello world'
```

```
string_2 = '5'  
string_3 = 'cinco'
```

```
string_1.capitalize() # Pone solo la primera letra en mayúscula ('Hello world')  
string_1.upper()      # Convierte TODO a mayúsculas ('HELLO WORLD')  
string_1.lower()      # Convierte TODO a minúsculas  
string_1.title()      # Pone mayúscula a cada palabra ('Hello World')  
string_1.count('l')   # Cuenta cuántas veces aparece la letra 'l'
```

Dividiendo y Cortando Textos

La función `len()` (de *length*) nos dice cuántos caracteres tiene el texto, ¡incluyendo espacios!

```
Python  
len(string_1)  
  
# .split() "corta" el texto. Por defecto lo corta donde hay espacios y nos devuelve una lista.  
string_1.split()  
  
# También podemos decirle que corte cada vez que encuentre la letra 'w'  
string_1.split('w')  
  
# .strip() actúa como una lija. Quita la basura (espacios o caracteres específicos) de los  
bordes izquierdo y derecho.  
string_4 = '(Hello World), '  
string_4.strip() # Quita espacios a los lados  
string_4.strip('(), ') # Quita paréntesis, comas y espacios de los bordes  
  
# .replace(viejo, nuevo) busca una palabra y la reemplaza por otra.  
string_5 = 'Hello World'  
string_5.replace('Hello', 'Hi')  
  
# .index() nos dice en qué posición (número) está una letra.  
# NOTA: ¡En Python siempre empezamos a contar desde el CERO (0)!  
string_4.index('o')  
string_4.index('o', 6) # Busca la letra 'o', pero empezando a buscar desde la posición 6
```

Slicing (Rebanando Textos)

Podemos extraer pedazos exactos usando corchetes [`inicio:fin:pasos`].

```
Python  
string_4[0] # Extrae el primer carácter (posición 0)
```

```
string_4[-2] # Los negativos cuentan desde el final hacia atrás. Extrae el penúltimo.  
string_4[1:-3] # Corta desde la posición 1 hasta la penúltima (sin incluir la -3)
```

Invertir un texto: El `::-1` significa "recorre todo de principio a fin, pero dando pasos hacia atrás"

```
string_4[::-1]
```

Llamadas en cadena y f-strings:

Podemos presionar varios "botones" a la vez leyendo de izquierda a derecha.

Las *f-strings* nos permiten inyectar variables directamente dentro de un texto usando llaves `{}`.

Python

Quita bordes -> convierte a mayúsculas -> divide en palabras

```
string_4.strip('()', ' ').upper().split()
```

```
nombre = 'Angel'
```

```
curso = 'Python for Data Engineering'
```

```
f'Hola {nombre}, este es el curso: {curso}' # La 'f' es obligatoria al inicio
```

Validaciones Automáticas:

Python

```
string_1.isdigit() # ¿Es todo números? Falso
```

```
string_2.isdigit() # ¿Es todo números? Verdadero ('5')
```

```
string_1.isalpha() # ¿Es todo letras? Falso (tiene un espacio)
```

```
string_3.isalpha() # ¿Es todo letras? Verdadero ('cinco')
```

'import' nos trae herramientas extra. 'string' trae el abecedario.

```
import string
```

```
string.ascii_lowercase
```

`.isalnum()` verifica si es alfanumérico (solo letras y/o números, sin caracteres especiales ni espacios)

```
string_1.isalnum()
```

```
string_2.isalnum()
```

```
string_3.isalnum()
```

3. Función PRINT: El Megáfono de Python

Si no usamos `print()`, Python calcula las cosas pero no nos las muestra en la pantalla final.

Existen "caracteres de escape" que dan formato:

- `\n`: Significa "New Line" (Crea un salto de línea, como presionar Enter).
- `\t`: Significa "Tab" (Da un espacio largo).

- La letra r antes del string (Raw string) le dice a Python que ignore el \n y lo imprima como texto normal (muy útil cuando copiamos rutas de archivos en Windows como C:\nueva_carpeta).

Python

```
print(string_4)
print('Cualquier cosa')
print(type(string_4))
```

Imprimiendo variables juntas

```
print('Mi nombre es:', nombre)
print(f'Mi nombre es: {nombre}')
```

Formateando la salida

```
print('Los datos de mi mascota son:\nNombre \t: Sugar\nRaza \t: Sharpei\nColor\t: Marron')
```

Raw string (Ignora el salto de línea)

```
print(r'Los datos de mi mascota son:\nNombre \t: Sugar\nRaza \t: Sharpei\nColor\t: Marron')
```